

AMRITA VISHWA VIDYAPEETHAM, AMRITAPURI
B. Tech – Computer Science Engineering (Artificial Intelligence)
22AIE401 – Reinforcement Learning

Team No: 9

Title: CardChess-RL

Dhruv Nair	AM.SC.U4AIE23501
Ihsal Riyas	AM.SC.U4AIE23037
Dano Aby Alex	AM.SC.U4AIE23029

1. Problem Background

The development of chess playing artificial intelligence is one of the most mature fields within game AI; chess engines such as Stockfish rely on human-designed evaluation functions with extensive alpha-beta search, while AlphaZero and Ic0 learn the evaluation function and move policy through self-play alone. These algorithms all share one commonality in their structure, which is that the game rules are known prior to play, and each position is evaluated according to those immutable rules throughout the game.

A second body of literature on game AI addresses a different class of decisions: selecting from a small, randomly drawn menu, as in the process of drafting during collectible card games such as Hearthstone, Magic: The Gathering, or Legends of Code and Magic. This is a sequential decision problem in its own right, with a chosen option affecting subsequent decision making, a new random menu being drawn for the player at each step, and optimality depending in part on what is available in the future. Current approaches to this type of problem (e.g. Vieira et al., 2019) treat the drafted "deck" separately from the game played with it; that is, the decision does not alter the rules of the current match.

Card-Chess RL exists at the boundary of the two problems. Each round, there is a coin flip allowing one player to select one of three randomly selected "rule cards" from a communal, without replacement deck. The effects of the selected card then change the rules governing what is allowed in terms of possible moves for that turn, before returning to normal chess play for another round. Therefore, the problem facing the RL algorithm is not merely "play chess" or "draft an optimal set of cards" on their own, but rather to figure out which of three randomly presented rule changes would help it win – based on incomplete knowledge about the rest of the deck, of a temporary nature, and of immediate effect in the move search and evaluation.

None of the existing chess engines, chess variants platform or card game reinforcement learning systems tackles this problem explicitly. This section summarizes the study of the three independent lines of existing research which this project is based upon – self-play reinforcement learning for board games with fixed rules, sequential decision making under uncertainty explored in CCG drafting, and engines/platforms that allow playing several chess variants individually with fixed rules.

2. Problem Statement

Card-Chess RL is a two-player game of chess in which the rules concerning the legality of moves are not constant throughout the entire game. At the beginning of every turn, a coin is flipped and the player who

wins the flip becomes the picker. This player is then dealt three randomly drawn rule cards from a common deck and must choose one card out of the three presented. The chosen rule card modifies the legality of moves for this particular round only and symmetrically for both players. The round is followed by a return to regular chess, and a new flip occurs.

It results in a problem of decision-making that cannot be broken down into “play chess well” and “choose cards well” as separate problems. The value of a rule card is not absolute but dependent on the board position, the side whose turn it is, the side whose turn it will be next who stands to gain more out of the new set of moves, and what is left in the deck of cards for the future turn of the opponent. On the other hand, the value of a board position is now ambiguous within a fixed set of rules since the rules for that position are not fixed but stochastic.

The specific problem being tackled by this research project is the following: given an initial board state, three randomly generated rules cards, and partial information about the rest of the deck, to devise a card selection strategy that would maximize the chance of victory in the game, where the corresponding moves are evaluated by a common search engine. The learning procedure needs to find such a strategy without using any explicitly defined rules card evaluation functions, based solely on outcomes of games and irregular decision process, since the agent can only act on those rounds when the outcome of the coin toss makes it the picker.

3. Why Reinforcement Learning?

There is neither any possibility of solving the card selection problem using a supervised approach nor can a direct search approach solve it, which makes reinforcement learning the right framework to use for the following three reasons.

There is no labeled data at all. Card Chess RL is an entirely new version. There is no body of data collected by humans playing the game, there are no examples of good card selections provided with annotations, and there are no experts who have agreed on a value system for cards to be followed.

This is an instance of sparse reward and credit assignment is difficult. The effect of playing a particular card is not immediately evident when the card is played. What may seem like a materially good card may end up opening up moves for the opponent two moves down the line; a seemingly neutral card may serve to restrict the opponent’s king. The only clear indicator is the final outcome of the game. This is exactly the scenario that Reinforcement Learning excels in – passing a reward that occurs very rarely backwards in time along the chain of actions in order to assign credit.

The decision is sequential and stochastic, not a classification. There is no replacement of the cards in the deck, which means each decision changes the probability of future options for both players. The coin toss means that the agent doesn’t know when its turn will come and can’t decide on the timing of its actions. So a policy should be able to take into account both the game and the deck states and to withstand some probabilistic element in a deterministic skill game – a similar setting that was successfully addressed with self-play TD-learning and dice rolling in TD-Gammon. It would be impossible for a heuristic to cover all the possible card-to-position interactions and to be flexible enough to adapt with new cards.

Finally, RL is the right technique for solving the particular problem. Play of moves is sufficiently well solved problem to be delegated to classical search methods – RL is not needed to solve chess. But exactly where prior art provides no solution, we need RL: learning the value of a temporary self-imposed rule change.

4. Literature Review

#	Title / System	Author(s) & Year / Type	Summary	Relevance to Card-Chess RL
1	A General Reinforcement Learning Algorithm that Masters Chess, Shogi and Go through Self-Play (AlphaZero)	<i>Silver et al., 2018 — Science (Paper)</i>	Self-play RL system combining Monte Carlo Tree Search with a combined policy/value network to reach superhuman play in chess, shogi and Go without human game data, using only the rules of each game.	Establishes the self-play plus policy/value network paradigm Module 2 builds on. Assumes one fixed rule set for the whole game — offers no mechanism for a rule set that changes every round.
2	Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping	<i>Ng, Harada & Russell, 1999 — ICML (Paper)</i>	Proves that adding a potential-based shaping term $\gamma\Phi(s') - \Phi(s)$ to the reward leaves the MDP's optimal policy unchanged, giving a principled way to add dense feedback to a sparse terminal reward.	Directly underpins Module 2's reward-shaping formula (§4.4). This project adapts the formulation to variable-length gaps between the agent's decisions, caused by the coin-toss picker mechanic — a case the original paper does not consider.
3	Proximal Policy Optimization Algorithms	<i>Schulman et al., 2017 — arXiv / OpenAI (Paper)</i>	Introduces a clipped surrogate objective for policy-gradient methods, giving much of the stability of trust-region methods with a far simpler implementation.	Candidate training algorithm for Module 2's card-selection policy, preferred over plain REINFORCE for its robustness to hyperparameter choice within a limited project timeline.
4	Temporal Difference Learning and TD-Gammon	<i>Tesauro, 1995 — Communications of the ACM (Paper)</i>	Trains a neural network via self-play and TD(λ) to play backgammon at near-expert level, learning purely from game outcomes in a game that includes a stochastic dice-roll element.	Earliest strong demonstration that self-play RL can absorb a chance element embedded in an otherwise skill-based board game. The card draw in Card-Chess RL plays a structurally similar role to TD-Gammon's dice roll.
5	A Survey of Monte Carlo Tree Search Methods	<i>Browne et al., 2012 — IEEE Trans. Computational Intelligence and AI in Games (Paper)</i>	Surveys MCTS variants and enhancements across games with widely differing branching factors, information structure and time budgets.	Background reference for Module 3's search design, and for how other search methods handle a horizon at which the rules of the game may change — directly relevant to this project's ruleset-horizon fix.
6	DeepStack: Expert-Level Artificial Intelligence in Heads-Up No-Limit Poker	<i>Moravčík et al., 2017 — Science (Paper)</i>	Combines recursive reasoning, decomposition and self-play-learned value estimates to play a game of imperfect information	Relevant precedent for reasoning under partial knowledge of what comes next. Informs the expected-value fallback option this project proposes for search

5. Existing Systems / Products Study

This section reviews three existing products that represent the closest real-world precedents for the engine and platform this project builds.

Fairy-Stockfish. An engine based on Stockfish that generalizes the minimax/alpha-beta search process for dozens of chess-like games (e.g., Xiangqi, Crazyhouse, Janggi, etc.) by means of configurable rule files, while using only one search engine. This case confirms the correctness of the approach of using a single search/evaluation process in the context of a specific move set, used in Module 3. In all these games, however, the rules of the game are fixed beforehand, not during the game like in Card-Chess RL.

Leela Chess Zero (lc0). Community-based chess engine inspired by AlphaZero approach, trained via distributed self-play using MCTS and a policy/value neural network, achieving top engine performance without any explicit knowledge of the chess game. This engine is most similar to Module 2's self-play training process and current weights vs. current weights comparison idea without any cards or rules.

Lichess / Chess.com variant platforms. These have GUI support for regular chess as well as some fixed-rules variants (such as Chess960, Crazyhouse, Three-check, and King of the Hill). The minimum acceptable UX level for this problem is set by these platforms. The platforms do not support a variant in which the rules selected for the current play are randomly selected from among cards by the player.

6. Synthesis — the Gap This Project Addresses

In these ten systems, three different areas of prior art can be distinguished: (a) reinforcement learning of self-play agents for playing full-information games with a constant set of rules (AlphaZero, TD-Gammon, Leela Chess Zero); (b) sequential decision-making on choosing among a set of random options, which was studied in CCG drafting (Vieira, Chaimowicz & Tavares); and (c) engines and platforms for playing multiple distinct variants of chess with their own constant sets of rules (Fairy-Stockfish, Lichess/Chess.com). There is no work among papers reviewed above or among products mentioned in the state-of-the-art analysis that combines all three: an agent which has to make choices between a randomized set of rule modifications in the context of normal two-player board game with constant rules enforcement by the same evaluation engine.

Card-Chess RL occupies the spot at the point where no work has been done yet. It employs the technique of potential-based reward shaping (Ng, Harada & Russell, 1999) and a PPO algorithm (Schulman et al., 2017) in an AlphaZero style self-play cycle (Silver et al., 2018). However, the formula of reward shaping used is modified in accordance with the rounds and varying timesteps involved due to the presence of the card mechanic. Also, the approach of shared search/evaluation employed by engines such as Fairy-Stockfish is modified accordingly in our work.

7. Proposed Research Direction

The present project attempts to implement and test a reinforcement learning agent based on self-play algorithm, designed to select rule-cards for each round of a chess variation game with modifiable rules, combined with a search engine that will play according to whatever set of rules was selected at a certain point of time. The project consists of three modules.

Module 1 – Game engine and user interface. Implementation of Card-Chess: board, standard chess legal rules, communal deck of cards with non-replacement draw, coin toss rule card selector, rule modifying component that modifies legal moves generator for the entire round. This module will provide a clean game environment API (state, legal actions, step, terminal) in order to be able to use it for playing both reinforcement learning agent from Module 2 and search engine from Module 3 and drag-and-drop interface similar to Lichess and Chess.com variants UI.

Module 2 — Card-selection agent. The policy network is trained using self-play, based on an encoding of the board position, the three candidate cards, and the current state of the deck, to select one of the three candidate cards. For the training of this module, we choose to use PPO (Schulman et al., 2017), in contrast with the vanilla policy gradient method, owing to its robustness in the case of little hyperparameter optimization, using a self-play scheme of the AlphaZero variety with current-weights-versus-current-weights. Our contribution in this case lies in a novel shaping of the potential-based rewards in the context of variable-length intervals between agent actions: the classical $\gamma\Phi(s') - \Phi(s)$ shaping relies on an assumption that the agent takes action every time step, which is not the case in our coin-toss setup.

Module 3 — Search and Evaluation Move. A unified alpha-beta or MCTS search procedure that searches on the moves in the active legal move set, using the Fairy-Stockfish paradigm of having one search module parametrized with ruleset-specific heuristics. The challenge in this case is the horizon issue – the search procedure can only proceed until the end of the current round since the ruleset beyond the horizon hasn't been defined yet. Here we propose a fall back to expected values for the leaves, evaluated with respect to a distribution of potential future rulesets based on the cards left in the deck, similar to DeepStack's use of learned value estimates in its decomposed search.

Evaluation: The proposed policy will be benchmarked on win rate against both a random and a greedy policy of maximizing material with one-ply search in the move engine, keeping the move engine constant for all policies. This is a parallel experiment to the one performed by Vieira et al. in order to demonstrate the importance of draft strategy in CCG.